Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Алгоритмы и структуры данных

ОТЧЕТ
к семестровой работе
на тему:

РЕШЕНИЕ АЛГОРИТМИЧЕСКИХ ЗАДАЧ

БГУИР КП 6-05-0612-02 113 ПЗ

Студент Н.А. Халамов

Преподаватель М.А. Калугина

Минск 2026

1 БИНАРНЫЕ ПОИСКОВЫЕ ДЕРЕВЬЯ

1.1 Задача 27 Удаление отца среднего

Условие задачи

Найти все листья дерева, выбрать из них средний по значению и удалить (правым удалением) его отца (если дерево состоит из единственной вершины, то удаляем эту вершину). Выполнить прямой (левый) обход полученного дерева.

Входные данные

Последовательность целых чисел - ключи, которые последовательно вставляются в бинарное поисковое дерево (BST).

Выходные данные

Последовательность ключей в порядке прямого (левого) обхода, полученного после удаления дерева. Если дерево стало пустым - выводится пустой список.

Теоретическое описание алгоритма

Сначала строим бинарное поисковое дерево (BST) из входной последовательности: каждый ключ вставляем по стандартным правилам – если меньше текущего узла, идём влево, иначе вправо.

Затем обходом в глубину (DFS) находим все листья – вершины без потомков. Собираем их значения, сортируем по возрастанию и берём элемент с индексом (L-1)//2, где L – количество листьев. Это и есть «средний» лист.

Если дерево состоит из одной вершины (она же лист и корень), удаляем её целиком – дерево становится пустым. Иначе находим родителя среднего листа и выполняем правое удаление этого родителя. Правое удаление: если у вершины два потомка, заменяем её значение максимумом левого поддерева (самой правой вершиной слева), а сам максимум удаляем. Если потомок один – просто заменяем вершину этим потомком.

В конце выполняем прямой (левый) обход полученного дерева: корень, левое поддерево, правое поддерево. Полученная последовательность и есть ответ.

Оценка Сложности

Пусть n – количество вершин, h – высота дерева, L – число листьев.

Построение BST: вставка n ключей занимает $O(n \cdot h)$ времени (каждый ключ спускается на глубину до h). В худшем случае (вырожденное дерево) h = n, сложность $O(n^2)$. В среднем h = O(log n), сложность O(n log n). Память: O(n) на узлы + O(h) на стек рекурсии.

Поиск листьев: обход дерева за O(n) времени, память O(h) на стек + O(L) на список листьев.

Сортировка листьев: O(L log L) времени, O(L) памяти.

Поиск отца и удаление: спуск по дереву на глубину h, время O(h), память O(h).

Прямой обход: O(n) времени, O(h) на стек + O(n) на результат.

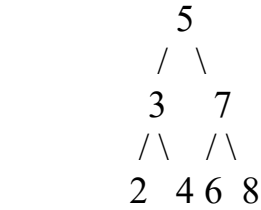
Итог: временная сложность $O(n \cdot h)$ (в худшем случае $O(n^2)$, в среднем O(n log n)), пространственная сложность O(n).

Пример работы

Тест 1: Полное дерево из 7 вершин

Вход: 5 3 7 2 4 6 8

Построенное дерево:



Шаг 1. Листья: {2, 4, 6, 8}.

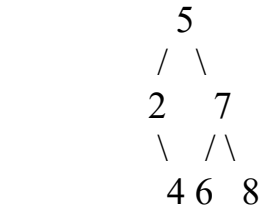
Шаг 2. Отсортированные: [2, 4, 6, 8].

Медиана (индекс (4-1)//2 = 1) это лист 4.

Шаг 3. Отец листа 4 - вершина 3.

Шаг 4. Правое удаление вершины 3: у 3 два потомка: left=2, right=4; максимум левого поддерева ({2}) = 2, его родителя нет. Заменяем ключ 3 на 2, левый потомок нового 2 становится None, правый остаётся 4.

Дерево после удаления:



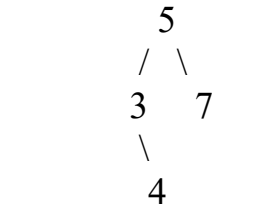
Шаг 5. Прямой обход: 5 - 2 - 4 - 7 - 6 - 8.

Выход: [5, 2, 4, 7, 6, 8]

Тест 2: Отец удаляемой вершины имеет одного потомка

Вход: 5 3 7 4

Построенное дерево:

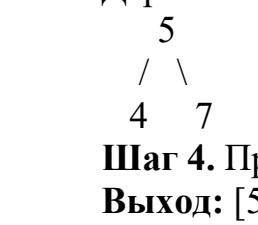


Шаг 1. Листья: {4, 7}. Отсортированные: [4, 7]. Медиана (индекс 0) это лист 4.

Шаг 2. Отец листа 4 это вершина 3.

Шаг 3. Правое удаление вершины 3: у 3 только правый потомок 4 (левого нет). Вершина 3 заменяется своим правым потомком 4.

Дерево после удаления:



Шаг 4. Прямой обход: 5 - 4 - 7.

Выход: [5, 4, 7]

1.2 Задача 28 Удаление средней с равными поддеревьями

Условие задачи

Найти и удалить (правым удалением) среднюю по значению вершину дерева, у которой высоты поддеревьев равны (листья дерева рассматриваются в качестве вершин, у которых высоты поддеревьев равны). Выполнить прямой (левый) обход полученного дерева.

Входные данные

Последовательность целых чисел – ключи, которые последовательно вставляются в бинарное поисковое дерево (BST).

Выходные данные

Последовательность ключей в порядке прямого (левого) обхода полученного после удаления дерева. Если дерево стало пустым – выводится пустой список.

Теоретическое описание алгоритма

Сначала строим бинарное поисковое дерево (BST) из входной последовательности: каждый ключ вставляем по стандартным правилам – если меньше текущего узла, идём влево, иначе вправо.

Для каждой вершины вычисляем высоты левого и правого поддеревьев. Высота пустого поддерева равна -1, высота листа – 0, высота произвольной вершины – единица плюс максимум высот её поддеревьев. Обходом дерева собираем все вершины, у которых высота левого поддерева равна высоте правого (листья автоматически попадают в этот список, так как у них оба поддерева пусты и имеют высоту -1).

Собираем значения подходящих вершин, сортируем по возрастанию и берём элемент с индексом (K-1)//2, где K – количество найденных вершин.

Выполняем правое удаление этой вершины. Если у неё два потомка, заменяем её значение максимумом левого поддерева (самой правой вершиной слева), а сам максимум удаляем. Если потомок один – заменяем вершину этим потомком, если потомков нет – удаляем целиком.

В конце выполняем прямой (левый) обход полученного дерева: корень, левое поддерево, правое поддерево. Полученная последовательность и есть ответ.

Оценка Сложности

Пусть n – количество вершин в дереве, h – его высота, K – количество вершин с равными высотами поддеревьев.

Построение BST требует $O(n \cdot h)$ времени, так как каждый из n ключей спускается от корня до места вставки на глубину до h. В худшем случае (вырожденное дерево) $h = n$ и сложность $O(n^2)$, в среднем $h = O(\log n)$ и сложность $O(n \log n)$.

Вычисление высот и сбор подходящих вершин выполняются за $O(n)$ времени одним рекурсивным обходом. Сортировка найденных вершин занимает $O(K \log K) \leq O(n \log n)$. Правое удаление требует спуска по дереву на глубину до h, то есть $O(h)$ времени. Прямой обход проходит все вершины за $O(n)$.

Итоговая временная сложность – $O(n \cdot h)$, то есть $O(n^2)$ в худшем случае и $O(n \log n)$ в среднем. Пространственная сложность – $O(n)$, определяется хранением дерева, списка подходящих вершин и результирующего списка.

Пример работы

Тест 1: Полное дерево из 7 вершин

Вход: 5 3 7 2 4 6 8

Построенное дерево:

```
      5
    /   \
   3     7
  / \   / \
 2   4 6   8
```

Шаг 1. Вычисляем высоты поддеревьев для каждой вершины:

Вершина 2: левое поддерево высота -1, правое -1 (равны)

Вершина 4: левое поддерево высота -1, правое -1 (равны)

Вершина 6: левое поддерево высота -1, правое -1 (равны)

Вершина 8: левое поддерево высота -1, правое -1 (равны)

Вершина 3: левое поддерево высота 0, правое 0 (равны)

Вершина 7: левое поддерево высота 0, правое 0 (равны)

Вершина 5: левое поддерево высота 1, правое 1 (равны)

Шаг 2. Все вершины имеют равные высоты поддеревьев: {2, 3, 4, 5, 6, 7, 8}.

Шаг 3. Отсортированные: [2, 3, 4, 5, 6, 7, 8]. Медиана (индекс (7-1)//2 = 3) это вершина 5.

Шаг 4. Правое удаление вершины 5:

У 5 два потомка: left=3, right=7.

Максимум левого поддерева ({3, 2, 4}) = 4, его родитель – 3.

Заменяем ключ 5 на 4, правый потомок 3 становится None.

Дерево после удаления:

```
      4
    /   \
   3     7
  /     / \
 2     6   8
```

Шаг 5. Прямой обход: 4, 3, 2, 7, 6, 8.

Выход: [4, 3, 2, 7, 6, 8]

Тест 2: Несимметричное дерево

Вход: 5 3 7 2

Построенное дерево:

```
    5
   / \
  3   7
 /
2
```

Шаг 1. Вычисляем высоты поддеревьев:

Вершина 2: высоты -1 и -1 (равны)

Вершина 3: левое высота 0, правое -1 (не равны)

Вершина 7: высоты -1 и -1 (равны)

Вершина 5: левое высота 1, правое 0 (не равны)

Шаг 2. Подходящие вершины: {2, 7}.

Шаг 3. Отсортированные: [2, 7]. Медиана (индекс 0) – вершина 2.

Шаг 4. Правое удаление вершины 2: у неё нет потомков, удаляем целиком.

Дерево после удаления:

```
    5
   / \
  3   7
```

Шаг 5. Прямой обход: 5, 3, 7.

Выход: [5, 3, 7]

1.3 Задача 10 Пути максимальной длины

Условие задачи

Найти все вершины дерева, через которые проходит наибольшее количество путей максимальной длины, и удалить их (правым удалением). Выполнить прямой (левый) обход полученного дерева.

Входные данные

Последовательность целых чисел – ключи, которые последовательно вставляются в бинарное поисковое дерево (BST).

Выходные данные

Последовательность ключей в порядке прямого (левого) обхода (pre-order: корень, левое поддерево, правое поддерево) полученного после удаления дерева. Если дерево стало пустым – выводится пустой список.

Теоретическое описание алгоритма

Рекурсивно вычисляем высоты левого и правого поддеревьев для каждой вершины. Диаметр дерева – максимум по всем вершинам суммы высот левого и правого поддеревьев. Для дерева из одной вершины диаметр равен нулю.

Находим все пути максимальной длины (равные диаметру). Для каждой вершины рассматриваем три случая: путь через неё из левого поддерева в правое, от неё до листа слева и от неё до листа справа. Если длина пути равна диаметру, находим все конечные листья на нужной глубине и для каждой пары концов строим полный путь через их общего предка. Все вершины на найденных путях учитываем в счётчике, который для каждой вершины накапливает количество проходящих через неё путей максимальной длины.

Отбираем вершины с максимальным значением счётчика и удаляем их правым удалением в порядке возрастания ключей. Правое удаление: если два потомка – заменяем вершину максимумом левого поддерева и удаляем этот

максимум; если один потомок – заменяем им; если потомков нет – удаляем целиком. После каждого удаления обновляем структуру дерева и ссылки на родителей.

В конце выполняем прямой (левый) обход: корень, левое поддерево, правое поддерево. Полученная последовательность и есть ответ.

Оценка Сложности

Пусть n – количество вершин, h – высота дерева, P – количество путей максимальной длины, V – суммарное количество вершин на всех таких путях, k – количество удаляемых вершин.

Построение BST со ссылками на родителей требует $O(n \cdot h)$ времени: каждый из n ключей спускается на глубину до h. В худшем случае $h = n$ и сложность $O(n^2)$, в среднем $h = O(\log n)$ и сложность $O(n \log n)$.

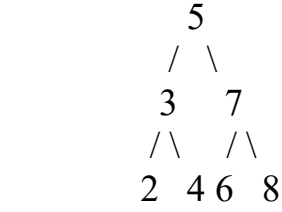
Вычисление высот выполняется рекурсивным обходом за O(n) времени. Поиск всех путей максимальной длины требует обхода всех вершин и для каждой вершины, через которую может проходить путь длины D, поиска конечных вершин на заданной глубине – в худшем случае $O(V \cdot h)$ времени. Сортировка ключей для удаления занимает $O(k \log k)$, а последовательное правое удаление k вершин – $O(k \cdot h)$. Прямой обход проходит все оставшиеся вершины за O(n).

Пример работы

Тест 1: Полное дерево из 7 вершин

Вход: 5 3 7 2 4 6 8

Построенное дерево:



Шаг 1. Вычисляем высоты поддеревьев и диаметр:

Вершины 2, 4, 6, 8: высоты поддеревьев (0, 0)

Вершина 3: высоты (1, 1), сумма = 2

Вершина 7: высоты (1, 1), сумма = 2

Вершина 5: высоты (2, 2), сумма = 4

Диаметр D = 4.

Шаг 2. Находим все пути длины 4:

2-3-5-7-6

2-3-5-7-8

4-3-5-7-6

4-3-5-7-8

Шаг 3. Подсчитываем количество путей через каждую вершину:

Вершина 2: 2 пути; вершина 3: 4 пути; вершина 4: 2 пути; вершина 5: 4 пути; вершина 6: 2 пути; вершина 7: 4 пути; вершина 8: 2 пути.

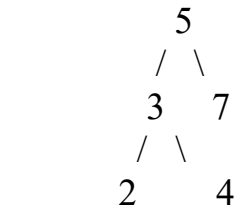
Максимум: 4 пути через вершины 3, 5, 7.

Шаг 4. Удаляем вершины 3, 5, 7 в порядке возрастания.

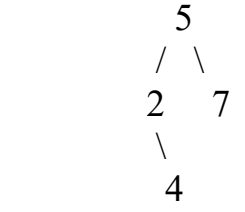
Шаг 5. Прямой обход: 4, 2, 6, 8.

Выход: [4, 2, 6, 8]

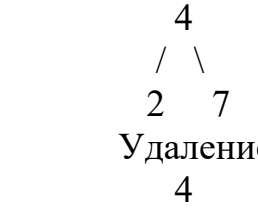
Тест 2: Несимметричное дерево
Вход: 5 3 7 2 4
Построенное дерево:



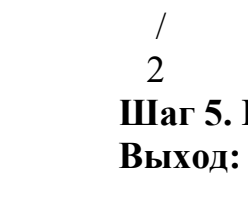
Шаг 1. Вычисляем высоты и диаметр:
Вершины 2, 4: высоты (0, 0)
Вершина 3: высоты (1, 1), сумма = 2
Вершина 7: высоты (0, 0)
Вершина 5: высоты (2, 1), сумма = 3
Диаметр D = 3.
Шаг 2. Находим все пути длины 3:
2-3-5-7
4-3-5-7
Шаг 3. Подсчитываем количество путей:
Вершина 2: 1 путь; вершина 3: 2 пути; вершина 4: 1 путь; вершина 5: 2 пути; вершина 7: 2 пути.
Максимум: 2 пути через вершины 3, 5, 7.
Шаг 4. Удаляем вершины 3, 5, 7.
Удаление 3: заменяем на 2.



Удаление 5: заменяем на 4.



Удаление 7: удаляем целиком.



Шаг 5. Прямой обход: 4, 2.
Выход: [4, 2]

2 РАЗРАБОТКА ЭФФЕКТИВНЫХ АЛГОРИТМОВ

2.1 Задача 24 Блок

Условие задачи

В таблице размера n на m, состоящей из 0 и 1, необходимо найти квадратный блок максимального размера, состоящий только из 1.

Входные данные

Два целых числа n и m – размеры таблицы (количество строк и столбцов), а также сама таблица, заданная n строками по m чисел (0 или 1) в каждой.

Выходные данные

Две пары координат – верхняя левая и нижняя правая клетки максимального квадратного блока из единиц. Если в таблице нет ни одной единицы, выводится сообщение об отсутствии такого блока.

Теоретическое описание алгоритма

Задача решается методом динамического программирования. Вводится вспомогательная матрица dp того же размера, что и исходная таблица, где значение dp[i][j] равно стороне максимального квадратного блока из единиц, правый нижний угол которого находится в клетке (i, j).

Базовым случаем являются клетки первой строки и первого столбца: для них значение dp[i][j] совпадает со значением исходной таблицы, так как квадрат большего размера, заканчивающийся в этих клетках, невозможен из-за выхода за границы таблицы.

Для всех остальных клеток, если в исходной таблице в позиции (i, j) стоит 0, то dp[i][j] = 0, поскольку квадрат из единиц не может заканчиваться в нулевой клетке. Если же в клетке стоит 1, то dp[i][j] вычисляется как единица плюс минимум из трёх значений: dp[i-1][j] (максимальный квадрат, заканчивающийся в клетке сверху), dp[i][j-1] (максимальный квадрат, заканчивающийся в клетке слева) и dp[i-1][j-1] (максимальный квадрат, заканчивающийся в клетке по диагонали).

В процессе заполнения матрицы dp параллельно отслеживается максимальное найденное значение и координаты клетки, в которой оно достигнуто. Пусть это клетка (r, c) и максимальная сторона равна S. Тогда нижняя правая клетка искомого квадрата – это (r, c), а верхняя левая клетка вычисляется как (r - S + 1, c - S + 1). Если в таблице вообще нет единиц, то максимальная сторона равна нулю и блок не существует.

Оценка сложности

Пусть n – количество строк, m – количество столбцов таблицы.

Построение матрицы dp требует однократного прохода по всем n·m клеткам. В каждой клетке выполняется константное число операций – проверка значения и вычисление минимума из трёх соседей, поэтому временная сложность составляет O(n·m).

Пространственная сложность определяется хранением матрицы dp размера n·m и составляет O(n·m). Итоговая временная сложность – O(n·m), пространственная – O(n·m).

Пример работы

Тест 1: Таблица с квадратом 3×3

Вход:

4 5
1 0 1 1 1
1 1 1 1 1
0 1 1 1 1
1 1 1 1 1

Шаг 1. Инициализируем матрицу dp нулями того же размера.

Шаг 2. Заполняем первую строку и первый столбец: они совпадают с исходной таблицей.

Шаг 3. Заполняем остальные клетки по формуле dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]), если matrix[i][j] = 1:

dp[1][1]: matrix=1, min(1, 1, 1) + 1 = 2
dp[1][2]: matrix=1, min(0, 2, 0) + 1 = 1
dp[1][3]: matrix=1, min(0, 1, 0) + 1 = 1
dp[1][4]: matrix=1, min(0, 1, 0) + 1 = 1
dp[2][1]: matrix=1, min(0, 2, 0) + 1 = 1
dp[2][2]: matrix=1, min(2, 1, 1) + 1 = 2
dp[2][3]: matrix=1, min(1, 2, 1) + 1 = 2
dp[2][4]: matrix=1, min(1, 2, 1) + 1 = 2
dp[3][1]: matrix=1, min(1, 1, 0) + 1 = 1
dp[3][2]: matrix=1, min(1, 2, 2) + 1 = 2
dp[3][3]: matrix=1, min(2, 2, 2) + 1 = 3
dp[3][4]: matrix=1, min(2, 3, 2) + 1 = 3

Итоговая матрица dp:

1 0 0 0 0
1 2 1 1 1
0 1 2 2 2
1 1 2 3 3

Шаг 4. Максимум в dp равен 3, впервые достигается в клетке (3, 3) (0-based). Верхняя левая клетка: (3 - 3 + 1, 3 - 3 + 1) = (1, 1).

Выход: (1, 1) - (3, 3)

Тест 2: Таблица, полностью заполненная единицами

Вход:

3 3
1 1 1
1 1 1
1 1 1

Шаг 1. Заполняем первую строку и столбец единицами.

Шаг 2. Последовательно заполняем остальные клетки:

dp[1][1] = 1 + min(1, 1, 1) = 2
dp[1][2] = 1 + min(1, 2, 1) = 2
dp[2][1] = 1 + min(1, 2, 1) = 2

dp[2][2] = 1 + min(2, 2, 2) = 3

Итоговая матрица dp:

1 1 1

1 2 2

1 2 3

Шаг 3. Максимум равен 3 в клетке (3,3). Верхняя левая: (3 - 3 + 1, 3 - 3 + 1) = (1, 1).

Выход: (1, 1) - (3, 3).

2.2 Задача 25 Преобразование строк

Условие задачи

На вход подаются две символьные строки A и B. Необходимо преобразовать строку B в строку A с минимальным суммарным штрафом, который определяется следующим образом: штраф за удаление символа из строки B равен x баллов; штраф за вставку символа в строку A – y баллов; штраф за выполнение замены символа в строке B на любой другой символ – z баллов.

Входные данные

Две символьные строки A и B, а также три целых неотрицательных числа x, y, z – стоимости операций удаления, вставки и замены соответственно.

Выходные данные

Одно целое число – минимальный суммарный штраф, необходимый для преобразования строки B в строку A. Дополнительно может быть выведена последовательность операций, приводящая к этому штрафу.

Теоретическое описание алгоритма

Задача решается методом динамического программирования. Вводится таблица dp размера (len(B)+1) на (len(A)+1), где dp[i][j] – минимальная стоимость преобразования префикса строки B длины i в префикс строки A длины j.

Базовые случаи: dp[0][j] = j · y (преобразование пустой строки в префикс A длины j требует j вставок), dp[i][0] = i · x (преобразование префикса B длины i в пустую строку требует i удалений), dp[0][0] = 0.

Для остальных клеток рассматриваются три операции:

Удаление символа из B: dp[i-1][j] + x

Вставка символа в B: dp[i][j-1] + y

Замена символа: dp[i-1][j-1] + z (если символы совпадают, стоимость dp[i-1][j-1])

Значение dp[i][j] выбирается как минимум из допустимых вариантов. Таблица заполняется построчно слева направо. Ответ – значение dp[len(B)][len(A)]. При необходимости последовательность операций восстанавливается обратным проходом от (len(B), len(A)) к (0, 0).

Оценка сложности

Обозначим через n длину строки A и через m длину строки B.

Таблица динамического программирования имеет размер (m+1) на (n+1), и каждая её клетка вычисляется за константное время – выполняется сравнение символов и нахождение минимума из трёх или четырёх значений. Следовательно, общая временная сложность алгоритма составляет O(n·m).

Пространственная сложность в базовой реализации определяется хранением всей таблицы dp, что даёт O(n·m) дополнительной памяти. Если не требуется восстановление последовательности операций, память можно оптимизировать до O(min(n, m)), поскольку для вычисления очередной строки таблицы достаточно хранить только предыдущую строку и одно дополнительное значение. При необходимости восстановления пути хранения всей таблицы остаётся неизбежным.

Пример работы

Тест 1: Выбор между заменой и удалением+вставкой

Вход: A = "abc", B = "xyz", x = 2, y = 2, z = 3

Все символы строк различны, поэтому алгоритм должен выбрать оптимальную стратегию: либо три замены (стоимость 3·z = 9), либо три удаления и три вставки (стоимость 3·x + 3·y = 12).

Шаг 1. Инициализируем таблицу dp размера 4 на 4 (B длины 3, A длины 3). Заполняем нулевую строку и нулевой столбец:

	""	a	b	c
""	0	2	4	6
x	2			
y	4			
z	6			

Шаг 2. Заполняем остальные клетки по формуле:

	""	a	b	c
""	0	2	4	6
x	2	3	5	7
y	4	5	6	8
z	6	7	8	9

Шаг 3. Ответ: dp[3][3] = 9. Алгоритм выбрал три замены, каждая стоимостью z=3.

Выход: 9

Тест 2: "kitten" – "catty"

Вход: A = "catty", B = "kitten", x = 1, y = 1, z = 1

Строки существенно различаются: B длиннее A на один символ, совпадают только две буквы 't'. Алгоритм должен найти оптимальную комбинацию замен, удалений и вставок.

Шаг 1. Инициализируем таблицу dp размера 7 на 6 (B длины 6, A длины 5). Заполняем нулевую строку (вставки) и нулевой столбец (удаления).

Шаг 2. Заполняем таблицу по формуле.

Итоговая таблица:

	""	c	a	t	t	y
""	0	1	2	3	4	5

k	1	1	2	3	4	5
i	2	2	2	3	4	5
t	3	3	3	2	3	4
t	4	4	4	3	2	3
e	5	5	5	4	3	3
n	6	6	6	5	4	4

Шаг 3. Ответ: dp[6][5] = 4. Оптимальная последовательность преобразований:

1. Заменить 'k' на 'c' (стоимость 1)
2. Заменить 'i' на 'a' (стоимость 1)
3. 't' остаётся без изменений (совпадение)
4. 't' остаётся без изменений (совпадение)
5. Заменить 'e' на 'y' (стоимость 1)
6. Удалить 'n' (стоимость 1)

Итоговая стоимость: 3 замены + 1 удаление = 4.

Выход: 4

2.3 Задача 37 Почта

Условие задачи

Вдоль большой широкой дороги располагаются деревни. Дорога представляет собой ось с целочисленными координатами, а позиции каждой деревни соответствует ее координата (единственное целое число). Известно, что все деревни расположены в разных местах (нет двух деревень с одинаковой координатой). Расстояние между двумя координатами вычисляется как абсолютная разность их величин. Было решено построить почтовые отделения в деревнях, но не обязательно во всех. Будем считать, что координаты деревни и почтового отделения, построенного в этой деревне, совпадают. Для построения почты в деревнях координаты почтовых отделений должны быть выбраны таким образом, чтобы сумма расстояний от каждой деревни до ближайшего почтового отделения была минимальна. Необходимо по данным координатам деревень и количеству почтовых отделений вычислить минимальную сумму из всех возможных сумм расстояний между деревнями и их ближайшими почтовыми отделениями. Необходимо также определить соответствующие координаты почтовых отделений.

Входные данные

Целое число n – количество деревень;

Целое число k – количество почтовых отделений для строительства;

Последовательность из n различных целых чисел – координаты деревень.

Выходные данные

Одно целое число – минимальная суммарная стоимость (сумма расстояний от каждой деревни до ближайшего почтового отделения);

Список из k координат – позиции почтовых отделений,

обеспечивающие эту минимальную стоимость.

Теоретическое описание алгоритма

Задача решается методом динамического программирования и требует предварительной сортировки координат деревень по возрастанию, поскольку оптимальное разбиение всегда состоит из непрерывных отрезков – если бы два отрезка пересекались, их можно было бы перегруппировать без увеличения стоимости.

На первом этапе вычисляется вспомогательная таблица cost[i][j], хранящая минимальную стоимость обслуживания деревень с индексами от i до j включительно одним почтовым отделением. Известно, что оптимальное расположение единственного почтового отделения для отрезка деревень – это медиана их координат. Для нечётного количества деревень медиана единственна и находится в середине отсортированного списка; для чётного количества любая точка между двумя центральными элементами даёт одинаковую сумму расстояний, и в качестве почтового отделения выбирается одна из деревень (левая или правая медиана). Стоимость вычисляется как сумма абсолютных разностей координат всех деревень отрезка с медианой.

На втором этапе строится таблица динамического программирования dp[k][i], где dp[k][i] – минимальная стоимость обслуживания первых i деревень (с индексами от 0 до i-1) ровно k почтовыми отделениями. Базовым случаем является dp[1][i] = cost[0][i-1] – обслуживание первых i деревень одним отделением. Переход осуществляется по формуле: dp[k][i] = min по всем j от k-1 до i-1 (dp[k-1][j] + cost[j][i-1]), то есть последнее почтовое отделение обслуживает деревни от j до i-1, а предыдущие k-1 отделений оптимально обслуживают первые j деревень. Параллельно с заполнением таблицы ведётся массив parent[k][i], запоминающий значение j, при котором достигается минимум – это необходимо для восстановления ответа.

После заполнения таблицы ответом является значение dp[k][n]. Восстановление координат почтовых отделений производится обратным проходом: начиная с i = n и k, находим j = parent[k][i], тогда последнее почтовое отделение обслуживает деревни от j до i-1 и располагается в медиане этого отрезка. Затем переходим к состоянию (k-1, j) и повторяем процесс до тех пор, пока не будут найдены все k почтовых отделений.

Оценка сложности

Итоговая временная сложность алгоритма определяется наиболее затратным этапом – заполнением таблицы динамического программирования – и составляет $O(n^2 + k \cdot n^2)$, что эквивалентно $O(k \cdot n^2)$ при $k \geq 1$. Итоговая пространственная сложность складывается из хранения таблицы cost размера $O(n^2)$, таблиц dp и parent размера $O(k \cdot n)$, а также отсортированного массива координат $O(n)$, что в сумме даёт $O(n^2)$.

Пример работы

Тест 1: Восемь деревень, два почтовых отделения

Входные данные:

n = 8, k = 2

Координаты: 1 5 10 12 15 20 25 30

Шаг 1. Подготовка и вычисление таблицы cost. Координаты уже отсортированы. Перед запуском основного алгоритма динамического программирования необходимо заполнить вспомогательную таблицу cost[i][j]. Рассчитаем ключевые значения cost (индексы деревень от 0 до 7):

cost[0][4] (деревни 1, 5, 10, 12, 15): медиана 10. Стоимость = |1-10| + |5-10| + |10-10| + |12-10| + |15-10| = 9 + 5 + 0 + 2 + 5 = 21.

cost[5][7] (деревни 20, 25, 30): медиана 25. Стоимость = |20-25| + |25-25| + |30-25| = 5 + 0 + 5 = 10.

cost[0][7] (все 8 деревень): медиана 12 (или 15). Стоимость = 11+7+2+0+3+8+13+18 = 62. Аналогичным образом вычисляются стоимости для всех остальных возможных отрезков.

Шаг 2. Заполнение таблицы dp (базовый случай для k=1). Таблица dp[k][i] хранит минимальную стоимость обслуживания первых i деревень с помощью k отделений. Для базового случая k=1 значение dp[1][i] просто равно cost[0][i-1], то есть стоимости обслуживания всех первых i деревень одним отделением. Например: dp[1][1] = 0, dp[1][5] = cost[0][4] = 21, dp[1][8] = cost[0][7] = 62.

Шаг 3. Основной этап динамического программирования (для k=2). Теперь ищем оптимальное разбиение для двух отделений. Нам нужно найти финальное значение dp[2][8]. Для этого алгоритм перебирает все возможные точки разбиения j (от 1 до 7). Точка j означает, что первые j деревень обслуживаются первым отделением (их стоимость берется из dp[1][j]), а оставшиеся деревни с индексами от j до 7 обслуживаются вторым отделением (их стоимость берется из cost[j][7]).

Рассмотрим вычисление dp[2][8] на нескольких вариантах разбиения j:

При j=1: dp[1][1] + cost[1][7] = 0 + 48 = 48.

При j=3: dp[1][3] + cost[3][7] = 9 + 28 = 37.

При j=4: dp[1][4] + cost[4][7] = 16 + 20 = 36.

При j=5: dp[1][5] + cost[5][7] = 21 + 10 = 31.

При j=6: dp[1][6] + cost[6][7] = 31 + 5 = 36.

Алгоритм выбирает минимальное значение из всех перебранных вариантов. Минимум достигается при j=5 и равен 31. Следовательно, dp[2][8] = 31.

Шаг 4. Восстановление ответа. Минимальная суммарная стоимость равна 31. Чтобы определить координаты почтовых отделений, мы смотрим, при каком j был достигнут минимум для dp[2][8]. Это j=5. Это означает, что второе (последнее) отделение обслуживает деревни с индексами от 5 до 7 (координаты 20, 25, 30). Оптимальное место для него – медиана этого отрезка, то есть координата 25.

Затем мы переходим к состоянию dp[1][5], что означает обслуживание первых пяти деревень (с индексами от 0 до 4, координаты 1, 5, 10, 12, 15) первым отделением. Медиана этого отрезка – координата 10, это и будет место строительства первого отделения.

Выход:

Минимальная стоимость: 31

Координаты почтовых отделений: [10, 25].

3 СТРУКТУРЫ ДАННЫХ

3.1 Задача 7 Карточки

Условие задачи

Имеется n карточек черного и белого цвета, которые сложены в стопку. Карточки раскладываются на стол в одну линию следующим образом: первая кладется на стол, вторая под низ стопки, третья – на стол, четвертая – под низ стопки и т. д., пока все карточки не будут выложены на стол. Необходимо определить, каким должно быть начальное расположение карточек в стопке, чтобы разложенные на столе карточки чередовались по цвету: белая, черная, белая, черная и т. д. (выдать для заданного n последовательность из 0 и 1, которая определяет начальное расположение карточек; при этом цифра 0 соответствует белой карточке, а цифра 1 – черной).

Входные данные

Одно целое число n – количество карточек в стопке.

Выходные данные

Последовательность из n цифр (0 и 1), определяющая начальное расположение карточек в стопке сверху вниз, при котором разложенные на столе карточки будут чередоваться: 0, 1, 0, 1, ...

Теоретическое описание алгоритма

Задача решается моделированием процесса раскладывания карточек с использованием очереди. Ключевая идея состоит в том, чтобы проследить, какая позиция в начальной стопке окажется на какой позиции на столе, а затем присвоить каждой позиции в стопке нужный цвет.

На первом этапе создаётся очередь, содержащая индексы от 0 до n-1, которые соответствуют позициям карточек в начальной стопке (0 – верхняя карточка, n-1 – нижняя). Также создаётся массив отображения table_order длины n, который будет хранить, какой индекс из начальной стопки попал на какую позицию на столе.

Затем моделируется сам процесс раскладывания. На каждом шаге извлекается элемент из головы очереди – это индекс карточки, которая кладётся на стол. Ему присваивается очередная позиция на столе. Если очередь не пуста, следующий элемент извлекается из головы и помещается в хвост очереди (имитация перекладывания под низ стопки). Этот процесс повторяется до тех пор, пока очередь не опустеет. В результате получается отображение: для каждой позиции на столе известно, какой индекс из начальной стопки на ней оказался.

На заключительном этапе формируется ответ. Поскольку на столе карточки должны чередоваться как 0, 1, 0, 1, ..., то карточка, оказавшаяся на позиции p на столе (нумерация с 0), должна иметь цвет p mod 2. Зная, какой индекс начальной стопки оказался на позиции p, присваиваем этому индексу цвет p mod 2. Таким образом заполняется результирующий массив длины n, который и является ответом.

Оценка сложности

Итоговая временная сложность алгоритма составляет O(n), так как каждая карточка обрабатывается константное число раз в процессе моделирования с очередью. Итоговая пространственная сложность также составляет O(n) и определяется хранением очереди и вспомогательных массивов. Алгоритм является оптимальным по времени, поскольку необходимо обработать каждую из n карточек хотя бы один раз.

Пример работы

Тест 1: Четыре карточки

Вход: n = 4

Шаг 1. Инициализируем очередь индексов: [0, 1, 2, 3]. Массив отображения пуст.

Шаг 2. Моделируем процесс раскладывания:

Итерация 1: Извлекаем 0 из очереди – на стол (позиция 0). Очередь: [1, 2, 3]. Извлекаем 1 и помещаем в хвост. Очередь: [2, 3, 1].

Итерация 2: Извлекаем 2 из очереди – на стол (позиция 1). Очередь: [3, 1]. Извлекаем 3 и помещаем в хвост. Очередь: [1, 3].

Итерация 3: Извлекаем 1 из очереди – на стол (позиция 2). Очередь: [3]. Извлекаем 3 и помещаем в хвост. Очередь: [3].

Итерация 4: Извлекаем 3 из очереди – на стол (позиция 3). Очередь пуста.

Отображение «позиция на столе – индекс в стопке»: {0: 0, 1: 2, 2: 1, 3: 3}.

Шаг 3. Присваиваем цвета: позиция 0 – цвет 0 (белая), позиция 1 – цвет 1 (чёрная), позиция 2 – цвет 0, позиция 3 – цвет 1.

Заполняем результирующий массив: index 0 = 0, index 1 = 0, index 2 = 1, index 3 = 1.

Выход: [0, 0, 1, 1]

Тест 2: Шесть карточек

Вход: n = 6

Шаг 1. Инициализируем очередь: [0, 1, 2, 3, 4, 5].

Шаг 2. Моделируем процесс:

Итерация 1: Извлекаем 0 – стол (позиция 0). 1 в хвост. Очередь: [2, 3, 4, 5, 1].

Итерация 2: Извлекаем 2 – стол (позиция 1). 3 в хвост. Очередь: [4, 5, 1, 3].

Итерация 3: Извлекаем 4 – стол (позиция 2). 5 в хвост. Очередь: [1, 3, 5].

Итерация 4: Извлекаем 1 – стол (позиция 3). 3 в хвост. Очередь: [5, 3].

Итерация 5: Извлекаем 5 – стол (позиция 4). 3 в хвост. Очередь: [3].

Итерация 6: Извлекаем 3 – стол (позиция 5). Очередь пуста.

Отображение: {0: 0, 1: 2, 2: 4, 3: 1, 4: 5, 5: 3}.

Шаг 3. Присваиваем цвета:

Позиция 0 (цвет 0) – index 0 = 0

Позиция 1 (цвет 1) – index 2 = 1
Позиция 2 (цвет 0) – index 4 = 0
Позиция 3 (цвет 1) – index 1 = 1
Позиция 4 (цвет 0) – index 5 = 0
Позиция 5 (цвет 1) – index 3 = 1
Результат: index 0=0, 1=1, 2=1, 3=1, 4=0, 5=0.
Выход: [0, 1, 1, 1, 0, 0]

3.2 Задача 31 Химическая реакция

Условие задачи

Есть n различных химических веществ, которые пронумерованы целыми числами от 1 до n. Матрица A размером n×n. Элемент матрицы A[i,j] равен номеру того вещества, которое получится в результате химической реакции веществ i и j (если вещества не вступают в реакцию, то A[i,j]=0). Имеется пробирка, в которую последовательно добавляются химические вещества во время эксперимента. Вещество i вступает в химическую реакцию с веществом j, если находится в пробирке непосредственно над ним. Вещества, которые не вступают в реакцию друг с другом, в пробирке не смешиваются. Необходимо определить, какие вещества и в какой последовательности будут находиться в пробирке после окончания эксперимента (указать номера веществ, находящихся в пробирке, начиная от верхнего уровня и заканчивая самым нижним).

Входные данные

Целое число n – количество различных химических веществ;

Матрица A размером n×n, где A[i][j] – номер вещества, получающегося при реакции веществ i и j (0, если реакции нет);

Последовательность целых чисел – номера веществ, добавляемых в пробирку в порядке эксперимента (сверху вниз).

Выходные данные

Последовательность целых чисел – номера веществ в пробирке после окончания эксперимента, начиная с верхнего уровня и заканчивая самым нижним. Если пробирка пуста (все вещества прореагировали), выводится пустой список.

Теоретическое описание алгоритма

Пробирка по своей природе является стеком – структурой данных с доступом только к верхнему элементу и работой по принципу LIFO (последним пришёл – первым вышел). Поэтому алгоритм моделирования эксперимента естественным образом строится на использовании стека.

На каждом шаге в стек помещается очередное вещество из входной последовательности. Сразу после этого проверяется, может ли новое добавленное вещество вступить в реакцию с веществом, находящимся непосредственно под ним (то есть с предыдущим верхним элементом стека). Для этого из стека мысленно извлекаются два верхних элемента, и по матрице A определяется результат их реакции. Если A[i][j] ≠ 0, реакция

происходит: оба вещества исчезают, а вместо них в стек помещается продукт реакции. Этот новый продукт, оказавшись на вершине стека, снова проверяется на реакцию с веществом под ним – и так до тех пор, пока либо реакция не перестанет происходить, либо в стеке не останется менее двух элементов.

Если же A[i][j] = 0, реакции нет, и оба вещества остаются в стеке на своих местах. После этого алгоритм переходит к следующему веществу из входной последовательности.

Процесс повторяется до тех пор, пока не будут обработаны все вещества из входной последовательности. Содержимое стека после этого, прочитанное от вершины к основанию, и является ответом на задачу.

Оценка сложности

Итоговая временная сложность алгоритма составляет O(m), где m – количество веществ, добавляемых в пробирку, поскольку каждое вещество и каждый продукт реакции обрабатываются константное число раз при использовании стека. Итоговая пространственная сложность равна $O(n^2 + m)$ и определяется хранением матрицы реакций размера n на n и стека, содержащего не более m элементов одновременно. Алгоритм является линейным по длине входной последовательности и оптимальным по времени, так как каждое вещество необходимо обработать хотя бы один раз.

Пример работы

Тест 1: Цепная реакция с полным превращением

Вход:

n = 6

Матрица A:

```
   1 2 3 4 5 6
1: 0 3 0 0 0 0
2: 3 0 0 0 0 0
3: 0 0 4 0 0 0
4: 0 0 0 5 0 0
5: 0 0 0 0 6 0
6: 0 0 0 0 0 0
```

Последовательность: 1 2 3 4 5

Шаг 1. Инициализируем пустой стек.

Шаг 2. Обрабатываем вещество 1:

Помещаем 1 в стек. Стек: [1] (верх слева).

В стеке один элемент – проверять реакцию не с чем.

Шаг 3. Обрабатываем вещество 2:

Помещаем 2 в стек. Стек: [2, 1].

Проверяем реакцию двух верхних: A[2][1] = 3 – реакция.

Извлекаем 2 и 1, помещаем продукт 3. Стек: [3].

В стеке один элемент – проверка реакции невозможна.

Шаг 4. Обрабатываем вещество 3:

Помещаем 3 в стек. Стек: [3, 3].

Проверяем реакцию: A[3][3] = 4 – реакция.

Извлекаем оба 3, помещаем продукт 4. Стек: [4].
В стеке один элемент – проверка реакции невозможна.
Шаг 5. Обрабатываем вещество 4:
Помещаем 4 в стек. Стек: [4, 4].
Проверяем реакцию: A[4][4] = 5 – реакция.
Извлекаем оба 4, помещаем продукт 5. Стек: [5].
Шаг 6. Обрабатываем вещество 5:
Помещаем 5 в стек. Стек: [5, 5].
Проверяем реакцию: A[5][5] = 6 – реакция.
Извлекаем оба 5, помещаем продукт 6. Стек: [6].
В стеке один элемент – проверка реакции невозможна.
Шаг 7. Последовательность исчерпана.
Выход: [6]

Тест 2: Частичные реакции с несколькими веществами в пробирке
Вход:
n = 6
Матрица A:
   1 2 3 4 5 6
1: 0 3 0 0 0 0
2: 3 0 0 0 0 0
3: 0 0 0 5 0 0
4: 0 0 5 0 0 0
5: 0 0 0 0 0 0
6: 0 0 0 0 0 0
Последовательность: 1 2 6 4 3
Шаг 1. Инициализируем пустой стек.
Шаг 2. Обрабатываем вещество 1:
Помещаем 1 в стек. Стек: [1].
Проверка реакции невозможна.
Шаг 3. Обрабатываем вещество 2:
Помещаем 2 в стек. Стек: [2, 1].
Проверяем реакцию: A[2][1] = 3 – реакция.
Извлекаем 2 и 1, помещаем продукт 3. Стек: [3].
Шаг 4. Обрабатываем вещество 6:
Помещаем 6 в стек. Стек: [6, 3].
Проверяем реакцию: A[6][3] = 0 – реакции нет.
Оба вещества остаются в стеке.
Шаг 5. Обрабатываем вещество 4:
Помещаем 4 в стек. Стек: [4, 6, 3].
Проверяем реакцию двух верхних: A[4][6] = 0 – реакции нет.
Все три вещества остаются в стеке.
Шаг 6. Обрабатываем вещество 3:
Помещаем 3 в стек. Стек: [3, 4, 6, 3].
Проверяем реакцию двух верхних: A[3][4] = 5 – реакция.

Извлекаем 3 и 4, помещаем продукт 5. Стек: [5, 6, 3].

Проверяем реакцию новых двух верхних: A[5][6] = 0 – реакции нет.

Шаг 7. Последовательность исчерпана.

Выход: [5, 6, 3]

3.3 Задача 23 Замок

Условие задачи

Имеется замок, который разделен на комнаты (имеется по крайней мере две комнаты). Условно можно представить замок разделенным на n×m клеток и его план задать следующим образом:

m – число клеток в направлении с севера на юг ($m \leq 50$) и n – число клеток в направлении с запада на восток ($n \leq 50$);

каждая клетка замка будет описываться одним числом p ($0 \leq p \leq 15$); это число является суммой следующих чисел: 1, если клетка имеет западную стену, 2 – северную, 4 – восточную, 8 – южную (внутренняя стена принадлежит обеим клеткам).

Необходимо для данного замка определить количество комнат в нем, площадь наибольшей комнаты, а также определить, какую стенку в замке нужно разрушить, чтобы получить комнату наибольшей площади (выдать площадь получившейся комнаты).

Входные данные

Два целых числа n и m – количество клеток по горизонтали (запад-восток) и вертикали (север-юг);

Матрица m × n целых чисел от 0 до 15, описывающих стены каждой клетки.

Выходные данные

Количество комнат в замке;

Площадь наибольшей комнаты;

Площадь комнаты, которая получится после разрушения одной стены (максимальная возможная);

Координаты клетки и направление стены, которую нужно разрушить (в формате: строка, столбец, направление).

Теоретическое описание алгоритма

Задача решается в три этапа с использованием обхода в ширину (BFS) для работы с графом смежных клеток.

На первом этапе клетки замка рассматриваются как вершины графа, а рёбра между ними проводятся там, где нет стены. Для каждой клетки проверяются четыре возможных направления: если соответствующий бит в числе p не установлен (бит 1 – запад, 2 – север, 4 – восток, 8 – юг), то в этом направлении есть проход. Затем с помощью BFS находятся все компоненты связности – комнаты. Каждой комнате присваивается уникальный идентификатор, а также вычисляется её площадь (количество клеток в компоненте). Одновременно определяется площадь наибольшей комнаты.

На втором этапе перебираются все внутренние стены замка. Для

каждой стены проверяется, разделяет ли она две разные комнаты (то есть клетки по разные стороны стены имеют разные идентификаторы комнат). Если да, то разрушение этой стены объединит две комнаты, и площадь новой комнаты будет равна сумме площадей исходных.

На третьем этапе выбирается стена, разрушение которой даёт максимальную объединённую площадь. При наличии нескольких стен с одинаковым максимальным результатом выбирается та, которая расположена ближе к западу (меньший номер столбца), а при равенстве – ближе к югу (больший номер строки, так как строки нумеруются с севера на юг). Для каждой клетки в первую очередь проверяется западная стена, затем северная. Перебор клеток выполняется по столбцам слева направо, внутри каждого столбца – по строкам снизу вверх.

Оценка сложности

Итоговая временная сложность алгоритма составляет O(n·m), так как каждая клетка и каждая стена обрабатываются константное число раз. Итоговая пространственная сложность равна O(n·m) и определяется хранением матрицы идентификаторов комнат, очереди BFS и вспомогательных массивов. Алгоритм является линейным по размеру входа и оптимальным по времени, поскольку для решения задачи необходимо хотя бы один раз просмотреть все клетки замка.

Пример работы

Тест 1: Замок 3 на 3 с пятью комнатами

Вход:

n = 3, m = 3

11 14 11

 7 7 11

 1 4 3

Шаг 1. Декодируем стены каждой клетки:

(0,0)=11=1+2+8: З, С, Ю. Проход на В.

(0,1)=14=2+4+8: С, В, Ю. Проход на З.

(0,2)=11=1+2+8: З, С, Ю. Проход на В.

(1,0)=7=1+2+4: З, С, В. Проход на Ю.

(1,1)=7=1+2+4: З, С, В. Проход на Ю.

(1,2)=11=1+2+8: З, С, Ю. Проход на В.

(2,0)=1=1: З. Проходы на С, В, Ю.

(2,1)=4=4: В. Проходы на З, С, Ю.

(2,2)=3=1+2: З, С. Проходы на В, Ю.

Шаг 2. Находим комнаты с помощью BFS:

Комната 1: начинаем с (0,0). Проход на В – (0,1). Из (0,1) прохода на Ю нет (стена). **Площадь = 2.**

Комната 2: начинаем с (0,2). Прохода на Ю нет (стена). **Площадь = 1.**

Комната 3: начинаем с (1,0). Проход на Ю – (2,0). Из (2,0) проход на В – (2,1). Из (2,1) проход на С – (1,1). Из (1,1) прохода на В нет (стена). **Площадь = 4.**

Комната 4: начинаем с (1,2). Прохода на Ю нет (стена). **Площадь = 1.**

Комната 5: начинаем с (2,2). **Площадь = 1.**

Шаг 3. Количество комнат: 5. Максимальная площадь: 4.

Шаг 4. Перебираем стены в порядке: столбцы слева направо, строки снизу вверх, для каждой клетки – сначала западная, потом северная:

(2,1) западная: комнаты 3 и 3, одна комната. Пропускаем.

(2,2) западная: комнаты 3 и 5, сумма 4+1=5. Лучшая: 5.

(0,1) западная: комнаты 1 и 1, одна комната. Пропускаем.

(0,2) западная: комнаты 1 и 2, сумма 2+1=3.

(1,1) западная: комнаты 3 и 3, одна комната. Пропускаем.

(1,2) западная: комнаты 3 и 4, сумма 4+1=5. Не больше лучшей (такая же).

Шаг 5. Максимальная площадь после разрушения: 5. Стена: западная у клетки (2,2), в 1-индексации – строка 3, столбец 3.

Выход:

Количество комнат: 5

Площадь наибольшей комнаты: 4

Максимальная площадь после разрушения стены: 5

Разрушить стену: строка 3, столбец 3, направление west

Тест 2: Замок 2 на 2 с двумя комнатами

Вход:

n = 2, m = 2

11 6

7 5

Шаг 1. Декодируем стены:

(0,0)=11=1+2+8: З, С, Ю. Проход на В.

(0,1)=6=2+4: С, В. Проходы на З, Ю.

(1,0)=7=1+2+4: З, С, В. Проход на Ю.

(1,1)=5=1+4: З, В. Проходы на С, Ю.

Шаг 2. Находим комнаты:

Комната 1: начинаем с (0,0). Проход на В – (0,1). Из (0,1) проход на Ю – (1,1). Из (1,1) прохода на З нет (стена). **Площадь = 3.**

Комната 2: начинаем с (1,0). Прохода на С нет (стена), на В нет (стена). **Площадь = 1.**

Шаг 3. Количество комнат: 2. Максимальная площадь: 3.

Шаг 4. Перебираем стены:

(1,0) северная: комнаты 2 и 1, сумма 1+3=4. Лучшая: 4.

(1,1) западная: комнаты 1 и 2, сумма 3+1=4. Не больше лучшей.

Шаг 5. Максимальная площадь после разрушения: 4. Стена: северная у клетки (1,0), в 1-индексации – строка 2, столбец 1.

Выход:

Количество комнат: 2

Площадь наибольшей комнаты: 3

Максимальная площадь после разрушения стены: 4

Разрушить стену: строка 2, столбец 1, направление north